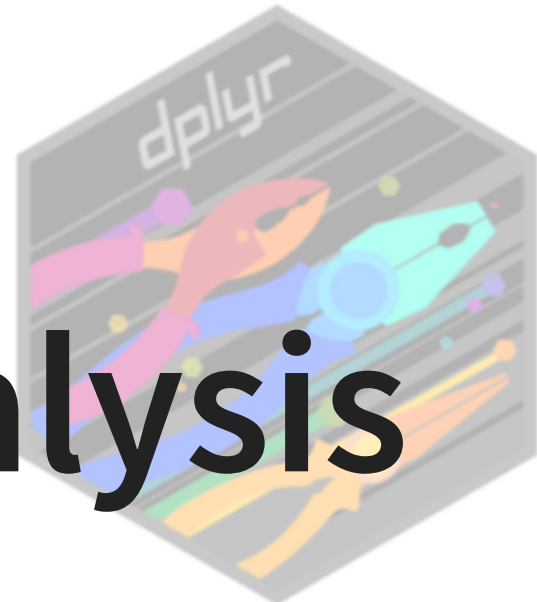
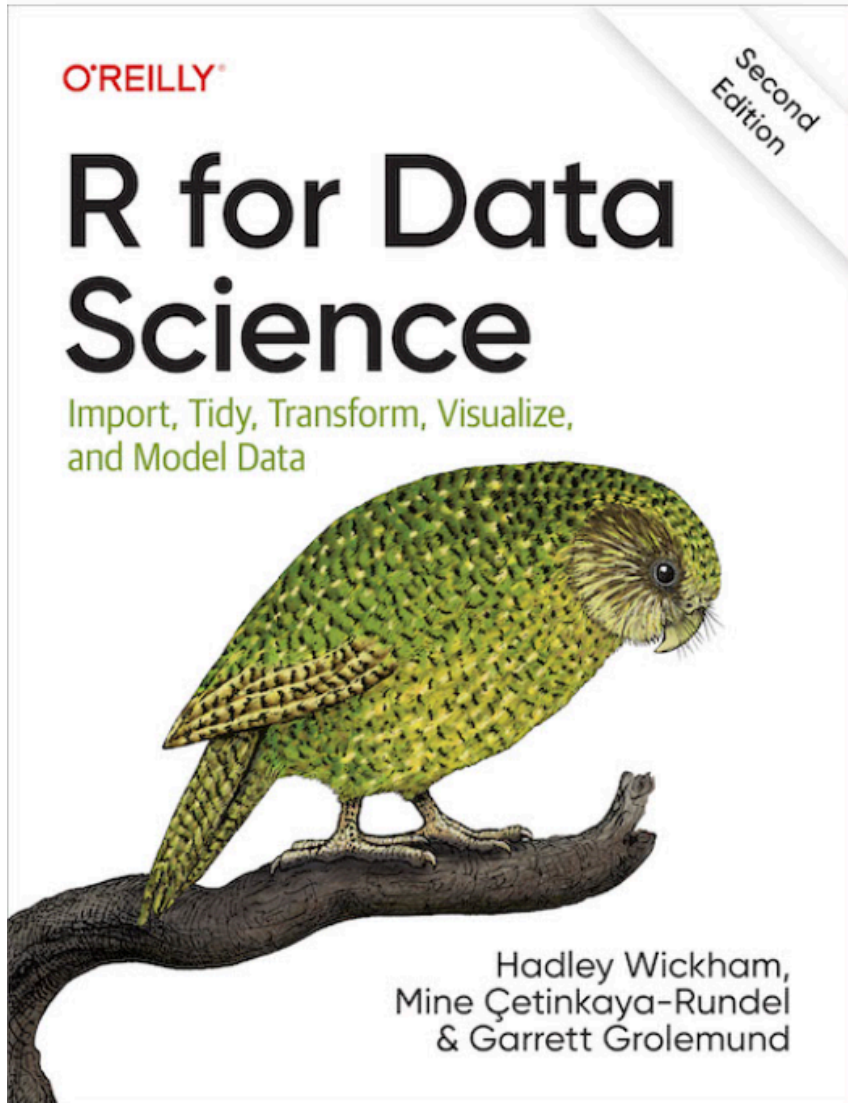




# Genomic Data Analysis in R

Blessing Olabosoye

2025-09-08



O'REILLY
Second Edition
R for Data Science
Import, Tidy, Transform, Visualize, and Model Data
Hadley Wickham,
Mine Çetinkaya-Rundel
& Garrett Grolemund

# Learning outcomes

- Understand the processes involved in data preprocessing, wrangling, and visualization
- Loading data with different packages, including data.table and readr
- Subsetting, merging, and handling missing values
- Grouping and summarizing results
- Functional programming
- Data visualization with ggplot2 package
- Publication-ready plots with cowplot

# Data Wrangling and Preprocessing

# Data wrangling

- This involves the process of loading the data, formatting it from its raw to a structured and readable form, and removing missing and undesirable data types (e.g NAN, NAs)
  - It also includes restructuring and reorganizing for downstream operations, and exploring to understand the data content as a whole or in part.
  - This is a more borader term for understanding the data

# Data preprocessing

- This is a subset of data wrangling that can also involves cleaning, removing missing values, but also deals with handling outliers, dimensional reduction, feature extraction, and feature engineering.
  - This ensures the needed variables are selected in the right form for the downstream analysis.

# Data Inputs and outputs

- Data inputs and outputs in R can take several forms in terms of file types. An example of data input and output includes:
  - Comma-separated value (.csv)
  - Tab-separated value (.tsv)
  - Microsoft Excel (.xlsx)
  - Plain text format (.txt)
  - R object files (.RDS, .RData)
  - N:B- Other extensions such as .vcf, .bam, .bim, etc can be read with .txt packages

# Data Inputs

| Functions | Description |
|---|---|
| read_* (csv or tsv)† | To read comma- or tab- separated files |
| read_delim()† | More general for both .csv and .tsv files |
| read_table()† | To read whitespace-separated files (.txt) |
| read_rds()† | To read a single R object .rds files |
| readRDS()‡ | To read a single R object .rds files |
| read_excel()‡‡ | To read any Excel files |
| read_* (xls or xlsx)‡‡ | To read .xls or .xlsx Excel file |
| fread()†† | Useful and fast for reading larger files |

†, ‡, ‡‡, and †† in *readr*, *base*, *readxl*, and *data.table* package, respectively

# Data Outputs

| Functions | Description |
|---|---|
| write_csv()† | To save comma separated files |
| write_tsv() † | To save tab separated files |
| write_delim()† | More general for both .csv and .tsv files |
| write_table()† | To save whitespace-separated files (.txt) |
| write_rds()† | To save a single R object .rds files |
| saveRDS()‡ | To save a single R object .rds files |
| save()‡ | To save one or more R object to .rds or .RData files |

† and ‡ in *readr* and *base* package, respectively

# Data Observation

To observe a data, i.e to see what's in it, the following functions becomes useful.

| Functions | Description |
|---|---|
| head()† | To see the first 6 rows and all columns |
| tail()† | To see the last 6 rows and all columns |
| view() ‡ /View()† | To view more rows and columns |
| summary()⋆ | To check the summary statistics |
| skim()†† | Like summary() but gives details about column variable types |

†, ‡, ⋆, and †† in *utils*, *tibble*, *base*, and *skimr* package, respectively

# Missing values

- As you beginning to work with your loaded data, you might notice either empty space, NA or NAN. Computing summary statistics or visualizing your data will result in a warning or an error.

- Missing values can either by 1) explicitly missing where you'll see NA or NAN, or 2) implicitly missing where the value is absent.

```r
library(tidyverse)
  irisc <- tibble(
  species = c("setosa", "setosa", "setosa", "setosa", "versicolor",
              "versicolor","versicolor"),
  plot = c(1,2,3,4,2,3,4),
  S.length   = c(5.1, 4.9, 4.7, NA, 5.0, 5.4, 4.6),
  S.width    = c(  3.5,    3.0,    NA,    3.1,    3.5,    3.9,    3.4))
irisc
```

```
# A tibble: 7 × 4
  species     plot S.length S.width
  <chr>      <dbl>    <dbl>   <dbl>
1 setosa         1      5.1     3.5
2 setosa         2      4.9     3
3 setosa         3      4.7    NA
4 setosa         4     NA       3.1
5 versicolor     2      5       3.5
6 versicolor     3      5.4     3.9
7 versicolor     4      4.6     3.4
```

- setosa has plot 4 explicitly missing sepal length value
- versicolor has plot 1 implicitly missing for sepal length value
- setosa has plot 3 explicitly missing for sepal width value

## To check for missing

- summary(): checks all columns with missing values
- is.na() or is.nan(): to check a column or rows

## To handle missing

- na.omit(): Excludes rows with missing values
- drop_na: Similar to na.omit() but useful for specific columns
- complete.cases(): Drops NA by subsetting

### Thoughts on missing values

- You want to understand the design of the missing values such as missing at random (MAR), missing completely at random (MCAR), or non missing at random (NMAR).
- Also, you want to know if removing is the best option or you want to impute the missing values using the mean or median values of either row(s) or column(s). See the *MICE* R package

summary()

```
   species                plot          S.length        S.width
 Length:7           Min.   :1.000   Min.   :4.600   Min.   :3.000
 Class :character   1st Qu.:2.000   1st Qu.:4.750   1st Qu.:3.175
 Mode  :character   Median :3.000   Median :4.950   Median :3.450
                    Mean   :2.714   Mean   :4.950   Mean   :3.400
                    3rd Qu.:3.500   3rd Qu.:5.075   3rd Qu.:3.500
                    Max.   :4.000   Max.   :5.400   Max.   :3.900
                                    NA's   :1       NA's   :1
```

na.omit()

```
# A tibble: 5 × 4
  species      plot S.length S.width
  <chr>       <dbl>    <dbl>   <dbl>
1 setosa          1      5.1     3.5
2 setosa          2      4.9     3
3 versicolor      2      5       3.5
4 versicolor      3      5.4     3.9
5 versicolor      4      4.6     3.4
```

drop_na()

```
# A tibble: 6 × 4
  species      plot S.length S.width
  <chr>       <dbl>    <dbl>   <dbl>
1 setosa          1      5.1     3.5
2 setosa          2      4.9     3
3 setosa          3      4.7    NA
4 versicolor      2      5       3.5
5 versicolor      3      5.4     3.9
6 versicolor      4      4.6     3.4
```

- complete.case()

```
# A tibble: 5 × 4
  species      plot S.length S.width
  <chr>       <dbl>    <dbl>   <dbl>
1 setosa          1      5.1     3.5
2 setosa          2      4.9     3
3 versicolor      2      5       3.5
4 versicolor      3      5.4     3.9
5 versicolor      4      4.6     3.4
```

```r
summary(irisc)
na.omit(irisc)
drop_na(irisc, S.length)
irisc[complete.case(irisc),]
```

# Data Transformation

Data transformation is a way to effectively and efficiently work with data frames. It makes downstream analysis seamless by structuring rows or columns in a way that fits the analysis

- Column-based
  - Relocating: *relocate()*
  - Grouping: *group_by()*
  - Mutating: *mutate()*
  - Renaming: *rename()*
  - Selecting: *select()*
- Row-based
  - Arranging: *arrange()*
  - Filtering: *filter()*
  - Distinct: *distinct()*
- Pivotting: Longer and wider
- Summarize: *summarise()*

## Arranging

This involves re-organizing variables by changing row orders based on certain column values. The function *arrange()* is used to re-organize in ascending order values of the column variable(s).

- To initialize in descending order, *desc()* can be added, i.e, **arrange(desc())**.
- Arrangement is controlled by the first variable if more that one variable is used.

## Relocating

This involves re-positioning column variables by moving their position. Using *relocate()*, variables are moved to the leftmost position by default except if *.before* or *.after* argument is provided.

## Filtering

Filtering involves choosing rows in a dataframe to keep based on the column values. The *filter()* function is used by specifying the condition for rows to keep and | or & is used to specify additional condition(s) to *or* or *and* consider, respectively.

## Selecting

Selecting involves choosing column variable of interest. The *select()* function is used and operator such as *:* and *!* can be used to indicate range and exclude, respectively.

## Distinct

Distinct helps to exclude duplicate row values in the data frame and only select unique values. The *distinct()* function is used.

## Mutating

Mutating involves creating a new column variable(s) based on existing one (s) or renaming an old column variable(s). The *mutate()* function is used and new columns are added to the rightmost position except if *.before* or *.after* argument is provided.

## Renaming

Renaming involves chnaging the existing column variable(s) names to a new ones either for efficiency or simplicity. The *rename()* function is used.

## Grouping

Grouping involves associating existing column variable(s) into groups for analysis. The *group_by()* function is used.

## Summarizing

Summarizing inlvolves obtaining the summary statistics of a grouped variable and output a single row value per group. This operation is performed by using the *summarise()*

## Pivoting

This involves changing or transposing variable from a wide format to a long format (more rows or lengthening) or from a long format to a wide format (more columns or widening). This operation is performed using the *pivot_longer()* or *pivot_wider()* function for lengthening or widening the data, respectively.

> **style**
>
> Piping is a way of expressing sequences of expressions and operations in a tidy manner. The operator %>% or |> is used.

# Strings

Sometimes you want to extract from or change certain strings in a data frame, the *str_⋆* function in the **Stringr** package can be used.

```
names(msleep)
```

```
 [1] "name"         "genus"        "vore"         "order"
"conservation"
 [6] "sleep_total"  "sleep_rem"    "sleep_cycle"  "awake"        "brainwt"
[11] "bodywt"
```

```
str_replace_all(names(msleep),"sleep_", "")
```

```
 [1] "name"         "genus"        "vore"         "order"
"conservation"
 [6] "total"        "rem"          "cycle"        "awake"        "brainwt"
[11] "bodywt"
```

```
str_count(names(msleep))
```

```
 [1]  4  5  4  5 12 11  9 11  5  7  6
```

Other string functions include: *str_detect()*, *str_split()*, *str_length()*, etc

# Data 1: Mammals sleep

```
data("msleep")
dim(msleep)
```

```
[1] 83 11
```

```
names(msleep)
```

```
 [1] "name"         "genus"        "vore"         "order"
"conservation"
 [6] "sleep_total"  "sleep_rem"    "sleep_cycle"  "awake"        "brainwt"
[11] "bodywt"
```

```
sleep_df <- msleep |> na.omit() |> select(-2)
names(sleep_df) <- str_replace_all(names(sleep_df),"sleep_", "")
dim(sleep_df)
```

```
[1] 20 10
```

```
sleep_df[1:3, c(1,2,5:10)]
```

```
# A tibble: 3 × 8
  name                        vore  total   rem cycle awake brainwt  bodywt
  <chr>                       <chr> <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>
1 Greater short-tailed shrew omni   14.9   2.3 0.133   9.1 0.00029   0.019
2 Cow                         herbi   4     0.7 0.667  20   0.423   600
3 Dog                         carni  10.1   2.9 0.333  13.9 0.07     14
```

```r
## Arranging
sleep_df |>
  arrange(total) |>
  select(-c(3,4)) |>
  head(5)
```

```
# A tibble: 5 × 8
  name                 vore    total   rem cycle awake brainwt  bodywt
  <chr>                <chr>   <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>
1 Horse                herbi     2.9   0.6 1      21.1  0.655  521
2 Cow                  herbi     4     0.7 0.667  20    0.423  600
3 Brazilian tapir      herbi     4.4   1   0.9    19.6  0.169  208.
4 Rabbit               herbi     8.4   0.9 0.417  15.6  0.0121   2.5
5 Eastern american mole insecti   8.4   2.1 0.167  15.6  0.0012   0.075
```

```r
## Fltering and selecting
sleep_df |>
  filter(order=="Rodentia") |>
  select(1,2,3,5:10) |>
  head(5)
```

```
# A tibble: 5 × 9
  name            vore  order    total   rem cycle awake brainwt bodywt
  <chr>           <chr> <chr>    <dbl> <dbl> <dbl> <dbl>   <dbl>  <dbl>
1 Guinea pig      herbi Rodentia   9.4   0.8 0.217  14.6  0.0055  0.728
2 Chinchilla      herbi Rodentia  12.5   1.5 0.117  11.5  0.0064  0.42
3 Golden hamster  herbi Rodentia  14.3   3.1 0.2     9.7  0.001   0.12
4 House mouse     herbi Rodentia  12.5   1.4 0.183  11.5  0.0004  0.022
5 Laboratory rat  herbi Rodentia  13     2.4 0.183  11    0.0019  0.32
```

```r
## Mutating
sleep_df |>
  mutate(sleep_nrem = (1-(rem/total))*total,
         bodywt_lbs = bodywt * 2.2) |>
  select(-c(2,3,4,9,10)) |>
  head(5)
```

```
# A tibble: 5 × 7
  name                          total   rem cycle awake sleep_nrem bodywt_lbs
  <chr>                         <dbl> <dbl> <dbl> <dbl>      <dbl>      <dbl>
1 Greater short-tailed shrew  14.9    2.3 0.133   9.1        12.6     0.0418
2 Cow                          4      0.7 0.667  20          3.3  1320
3 Dog                         10.1    2.9 0.333  13.9        7.2    30.8
4 Guinea pig                   9.4    0.8 0.217  14.6        8.6     1.60
5 Chinchilla                  12.5    1.5 0.117  11.5       11       0.924
```

```r
## Relocating
sleep_df |>
  relocate(brainwt, .after = bodywt) |>
  select(1,5:10) |>
  head(5)
```

```
# A tibble: 5 × 7
  name                        total   rem cycle awake  bodywt brainwt
  <chr>                       <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>
1 Greater short-tailed shrew  14.9   2.3 0.133   9.1   0.019 0.00029
2 Cow                          4     0.7 0.667  20   600     0.423
3 Dog                         10.1   2.9 0.333  13.9  14     0.07
4 Guinea pig                   9.4   0.8 0.217  14.6   0.728 0.0055
5 Chinchilla                  12.5   1.5 0.117  11.5   0.42  0.0064
```

```r
## Renaming
sleep_rename <- sleep_df |>
  rename(body_wt=bodywt)

sleep_rename[1:6, c(1,2,3,5,6,7,10)]
```

```
# A tibble: 6 × 7
  name                        vore  order          total   rem cycle body_wt
  <chr>                       <chr> <chr>          <dbl> <dbl> <dbl>   <dbl>
1 Greater short-tailed shrew omni  Soricomorpha    14.9   2.3 0.133   0.019
2 Cow                         herbi Artiodactyla     4     0.7 0.667 600
3 Dog                         carni Carnivora      10.1   2.9 0.333  14
4 Guinea pig                  herbi Rodentia        9.4   0.8 0.217   0.728
5 Chinchilla                  herbi Rodentia       12.5   1.5 0.117   0.42
6 Lesser short-tailed shrew   omni  Soricomorpha    9.1   1.4 0.15    0.005
```

```r
## Grouping and summarizing
sleep_df |>
  group_by(vore,conservation) |>
  summarise(avd=mean(bodywt), median=median(total)) |>
  head()
```

```
# A tibble: 6 × 4
# Groups:   vore [2]
  vore  conservation     avd median
  <chr> <chr>          <dbl>  <dbl>
1 carni domesticated   8.65    11.3
2 carni lc             3.5     17.4
3 herbi domesticated 225.       8.4
4 herbi en             0.12    14.3
5 herbi lc             0.211   13.4
6 herbi nt             0.022   12.5
```

# Data 2: WHO-TB

```r
data("who2")

who2[1:10, 1:7]
```

```
# A tibble: 10 × 7
   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp_m_4554
   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 Afghanistan  1980       NA        NA        NA        NA        NA
 2 Afghanistan  1981       NA        NA        NA        NA        NA
 3 Afghanistan  1982       NA        NA        NA        NA        NA
 4 Afghanistan  1983       NA        NA        NA        NA        NA
 5 Afghanistan  1984       NA        NA        NA        NA        NA
 6 Afghanistan  1985       NA        NA        NA        NA        NA
 7 Afghanistan  1986       NA        NA        NA        NA        NA
 8 Afghanistan  1987       NA        NA        NA        NA        NA
 9 Afghanistan  1988       NA        NA        NA        NA        NA
10 Afghanistan  1989       NA        NA        NA        NA        NA
```

```r
## Lengthening
who2 |>
    select("country","year", "sp_m_1524","sp_f_1524", "sn_m_1524", "sn_f_15
    na.omit() |>
    pivot_longer(names_to = "diagnosis",
                 cols = sp_m_1524:sn_f_1524,
                 values_to = "cases")|>head()
```

```
# A tibble: 6 × 4
  country  year diagnosis cases
  <chr>   <dbl> <chr>     <dbl>
1 Albania  2006 sp_m_1524    24
2 Albania  2006 sp_f_1524    12
3 Albania  2006 sn_m_1524     4
4 Albania  2006 sn_f_1524     6
5 Albania  2007 sp_m_1524    19
6 Albania  2007 sp_f_1524    13
```

# Data 3: Medicare & Medicaid services

```
data("cms_patient_care")
## Widening
head(cms_patient_care,4)
```

```
# A tibble: 4 × 5
  ccn    facility_name   measure_abbr       score type
  <chr>  <chr>           <chr>              <dbl> <chr>
1 011500 BAPTIST HOSPICE beliefs_addressed 202   denominator
2 011500 BAPTIST HOSPICE beliefs_addressed 100   observed
3 011500 BAPTIST HOSPICE composite_process 202   denominator
4 011500 BAPTIST HOSPICE composite_process  88.1 observed
```

```
cms_patient_care |>
  pivot_wider(names_from = type,
              values_from = score) |>
  head(3)
```

```
# A tibble: 3 × 5
  ccn    facility_name   measure_abbr       denominator observed
  <chr>  <chr>           <chr>                    <dbl>    <dbl>
1 011500 BAPTIST HOSPICE beliefs_addressed          202    100
2 011500 BAPTIST HOSPICE composite_process          202     88.1
3 011500 BAPTIST HOSPICE dyspena_treatment          110     99.1
```

# Merging

Merging allows the combination of variables from two data sets using a common key.

## Data 4: Women

```r
data("women")

head(women)
```

```
  height weight
1     58    115
2     59    117
3     60    120
4     61    123
5     62    126
6     63    129
```

```r
set.seed(232)
ID <- stringi::stri_rand_strings(n=15, length = 5)

## Using cbind
women_df <- cbind(women, ID) |> select(3,1,2)
head(women_df)
```

```
     ID height weight
1 xv6Re     58    115
2 tb4Vl     59    117
3 h4go7     60    120
4 6Z38H     61    123
5 XHcNn     62    126
6 Gwpu4     63    129
```

# Data 5: Plant Growth

```r
## Using inner join

bmi <- sprintf("%.3f", women$weight/(women$height^2))
bmi_w <- as.data.frame(cbind(ID, bmi))

women_df |>
  inner_join(bmi_w) |>
  head()
```

```
    ID height weight   bmi
1 xv6Re     58    115 0.034
2 tb4Vl     59    117 0.034
3 h4go7     60    120 0.033
4 6Z38H     61    123 0.033
5 XHcNn     62    126 0.033
6 Gwpu4     63    129 0.033
```

```r
data("PlantGrowth")

df1 <- data.frame(cbind(PlantGrowth[c(8:13,21:23), ],
                        rep=rep(1:3, times=3)), row.names = NULL)
df1
```

```
  weight group rep
1   4.53  ctrl   1
2   5.33  ctrl   2
3   5.14  ctrl   3
4   4.81  trt1   1
5   4.17  trt1   2
6   4.41  trt1   3
7   6.31  trt2   1
8   5.12  trt2   2
9   5.54  trt2   3
```

```r
df2 <- data.frame(height = rnorm(7, 75, 1),
                  fert = rnorm(7, 2),
                  group = c(rep("ctrl", times=2), rep("trt1", times=1),
                            rep("trt2", times=2), rep("trt3", times=2)),
                  rep=c(1,2,1,1,2,1,2))
```

```r
df2
```

| | height | fert | group | rep |
|---|---|---|---|---|
| 1 | 75.07747 | 1.801138 | ctrl | 1 |
| 2 | 74.74316 | 2.173368 | ctrl | 2 |
| 3 | 77.16210 | 2.109122 | trt1 | 1 |
| 4 | 75.70699 | 3.476592 | trt2 | 1 |
| 5 | 72.98554 | 2.046142 | trt2 | 2 |
| 6 | 74.85279 | 2.017269 | trt3 | 1 |
| 7 | 73.59630 | 2.049978 | trt3 | 2 |

```r
## Inner join

df1 |>
  inner_join(df2,
             by = c("group", "rep"
```

| | weight | group | rep | height | fert |
|---|---|---|---|---|---|
| 1 | 4.53 | ctrl | 1 | 75.07747 | 1.801138 |
| 2 | 5.33 | ctrl | 2 | 74.74316 | 2.173368 |
| 3 | 4.81 | trt1 | 1 | 77.16210 | 2.109122 |
| 4 | 6.31 | trt2 | 1 | 75.70699 | 3.476592 |
| 5 | 5.12 | trt2 | 2 | 72.98554 | 2.046142 |

```r
## Right join
df1 |>
  right_join(df2,
             by = c("group", "rep"
```

| | weight | group | rep | height | fert |
|---|---|---|---|---|---|
| 1 | 4.53 | ctrl | 1 | 75.07747 | 1.801138 |
| 2 | 5.33 | ctrl | 2 | 74.74316 | 2.173368 |
| 3 | 4.81 | trt1 | 1 | 77.16210 | 2.109122 |
| 4 | 6.31 | trt2 | 1 | 75.70699 | 3.476592 |
| 5 | 5.12 | trt2 | 2 | 72.98554 | 2.046142 |
| 6 | NA | trt3 | 1 | 74.85279 | 2.017269 |
| 7 | NA | trt3 | 2 | 73.59630 | 2.049978 |

```r
## Left join
df1 |>
  left_join(df2,
            by = c("group", "rep")
```

| | weight | group | rep | height | fert |
|---|---|---|---|---|---|
| 1 | 4.53 | ctrl | 1 | 75.07747 | 1.801138 |
| 2 | 5.33 | ctrl | 2 | 74.74316 | 2.173368 |
| 3 | 5.14 | ctrl | 3 | NA | NA |
| 4 | 4.81 | trt1 | 1 | 77.16210 | 2.109122 |
| 5 | 4.17 | trt1 | 2 | NA | NA |
| 6 | 4.41 | trt1 | 3 | NA | NA |
| 7 | 6.31 | trt2 | 1 | 75.70699 | 3.476592 |
| 8 | 5.12 | trt2 | 2 | 72.98554 | 2.046142 |
| 9 | 5.54 | trt2 | 3 | NA | NA |

```r
## Anti-join
df1 |>
  anti_join(df2, by = c("group", "rep"))
```

| | weight | group | rep |
|---|---|---|---|
| 1 | 5.14 | ctrl | 3 |
| 2 | 4.17 | trt1 | 2 |
| 3 | 4.41 | trt1 | 3 |
| 4 | 5.54 | trt2 | 3 |

```r
## Full join

df1 |>
  full_join(df2,
            by = c("group", "rep"))
```

```
   weight group rep   height     fert
1    4.53  ctrl   1 75.07747 1.801138
2    5.33  ctrl   2 74.74316 2.173368
3    5.14  ctrl   3       NA       NA
4    4.81  trt1   1 77.16210 2.109122
5    4.17  trt1   2       NA       NA
6    4.41  trt1   3       NA       NA
7    6.31  trt2   1 75.70699 3.476592
8    5.12  trt2   2 72.98554 2.046142
9    5.54  trt2   3       NA       NA
10     NA  trt3   1 74.85279 2.017269
11     NA  trt3   2 73.59630 2.049978
```

# Functional Programming

## Functions

Functions is an important operations in data analysis, especially in repetitive tasks, thereby, reducing incidental mistakes and repeatability. Although several functions are available in R, you might need to write yours.

```r
## Population SD:

sd_p1 <- function(x){sqrt(mean((x-mean(x))^2))}

sd.p2 <- function(x){sqrt((length(x)-1)/length(x))*sd(x)}

## Population variance

var_p1 <- function(x){mean((x-mean(x))^2)}

var.p2 <- function(x){(length(x)-1)/length(x)*var(x)}
```

# Iterations

Iteration involves repeatedly perform similar operations on different variables or objects. To ensure efficiency, it's good to use function to carry out iterative process.

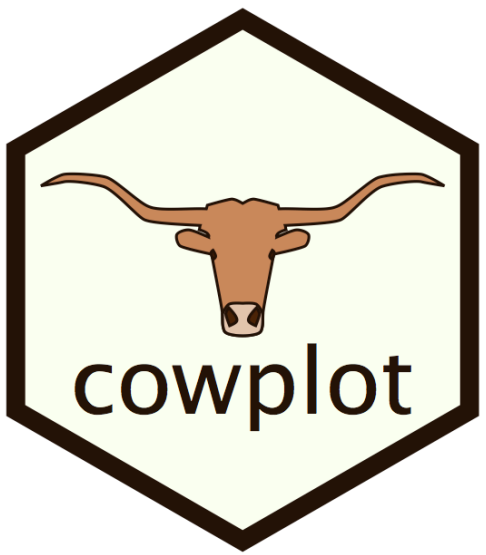
```
## Across

sleep_df|>group_by(conservation)|>summarise(across(where(is.numeric),
                                                  mean, na.rm=TRUE))
```

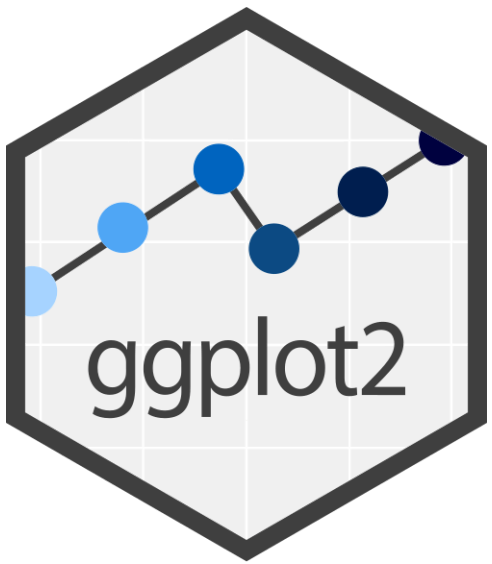
```
# A tibble: 5 × 7
  conservation total   rem cycle awake brainwt  bodywt
  <chr>        <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>
1 domesticated  8.61  1.62 0.458  15.4 0.172   154.
2 en           14.3   3.1  0.2     9.7 0.001     0.12
3 lc           13.8   3    0.219  10.2 0.00316   0.724
4 nt           12.5   1.4  0.183  11.5 0.0004    0.022
5 vu            4.4   1    0.9    19.6 0.169   208.
```

Other iterations includes:

- For loop - apply - sapply - lapply - mapply

# Data Visualization




**Quote**

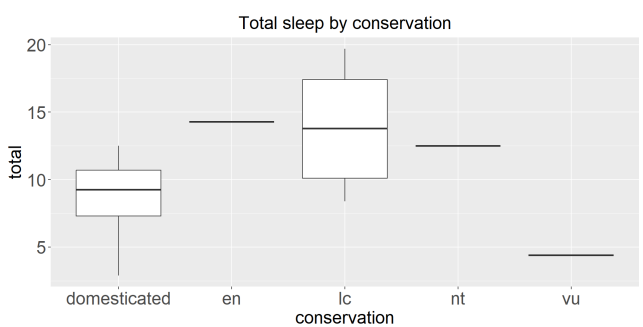
"The simple graph has brough more information to the data analyst's mind than any other device." - John Tukey

# Graphs

# The Grammar of graphics

The grammar of graphics implemented by the versatile package, *ggplot2*, for describing, building, and storytelling graphs in order to understand the data.

```
ggplot(sleep_df, aes(conservation, total))+
  geom_boxplot()+
  ggtitle("Total sleep by conservation")+ #Plot title
  theme(plot.title = element_text(hjust = 0.5, size=20),#centre plot title&
        axis.text = element_text(size = 20), #increase axis text size
        axis.title = element_text(size = 20), #increase axis title sze
        plot
        )
```



```r
library(data.table)

cowt <- "https://raw.githubusercontent.com/Bleznolap/GENET_5910_2025/main/D
cat_map <- fread(cowt)

dim(cat_map)
```

```
[1] 635732      4
```

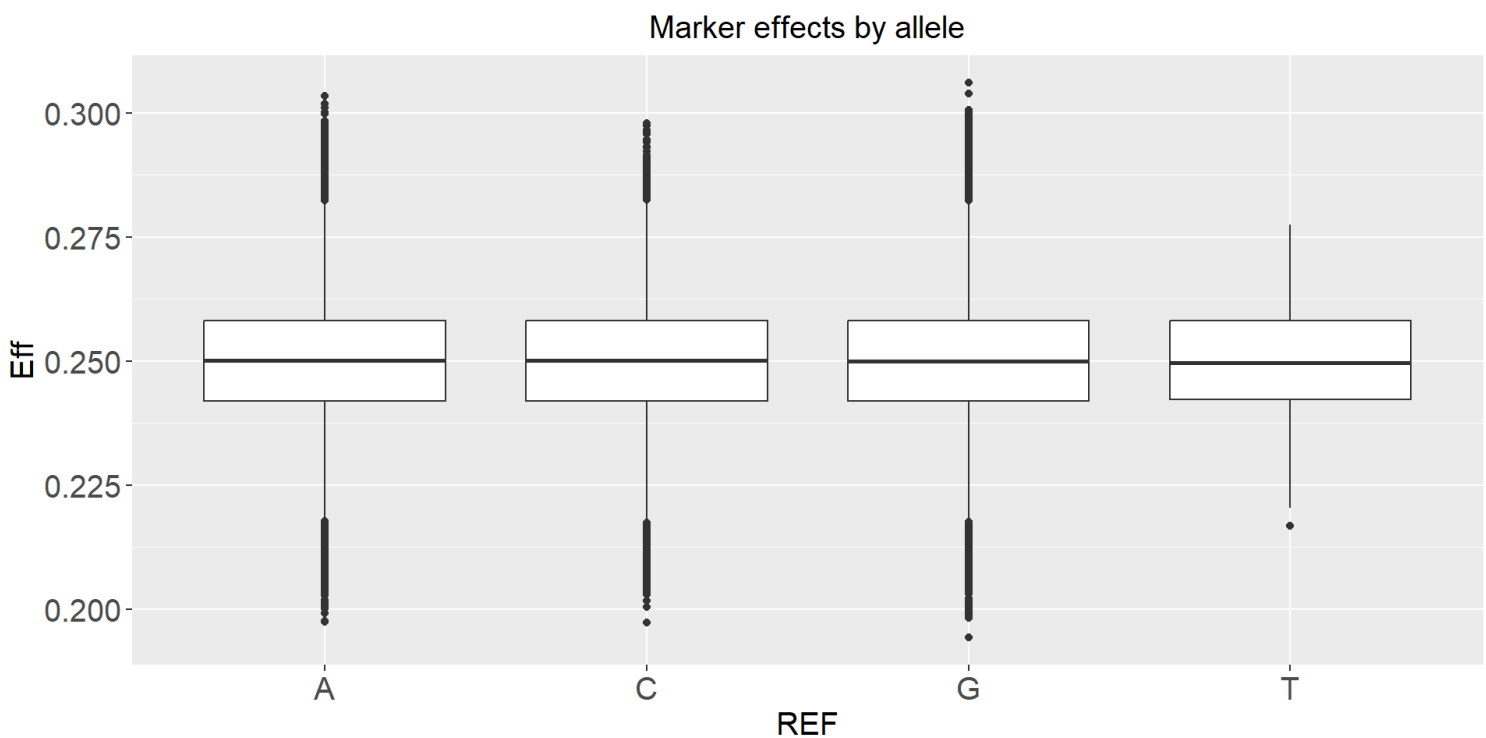
```r
head(cat_map)
```

```
      V1     V2     V3     V4
   <int>  <int>  <int> <char>
1:     1  16947  16947    C/A
2:     1  36337  36337    A/G
3:     1  78655  78655    A/G
4:     1  83412  83412    G/A
5:     1  89725  89725    A/G
6:     1 100260 100260    G/A
```

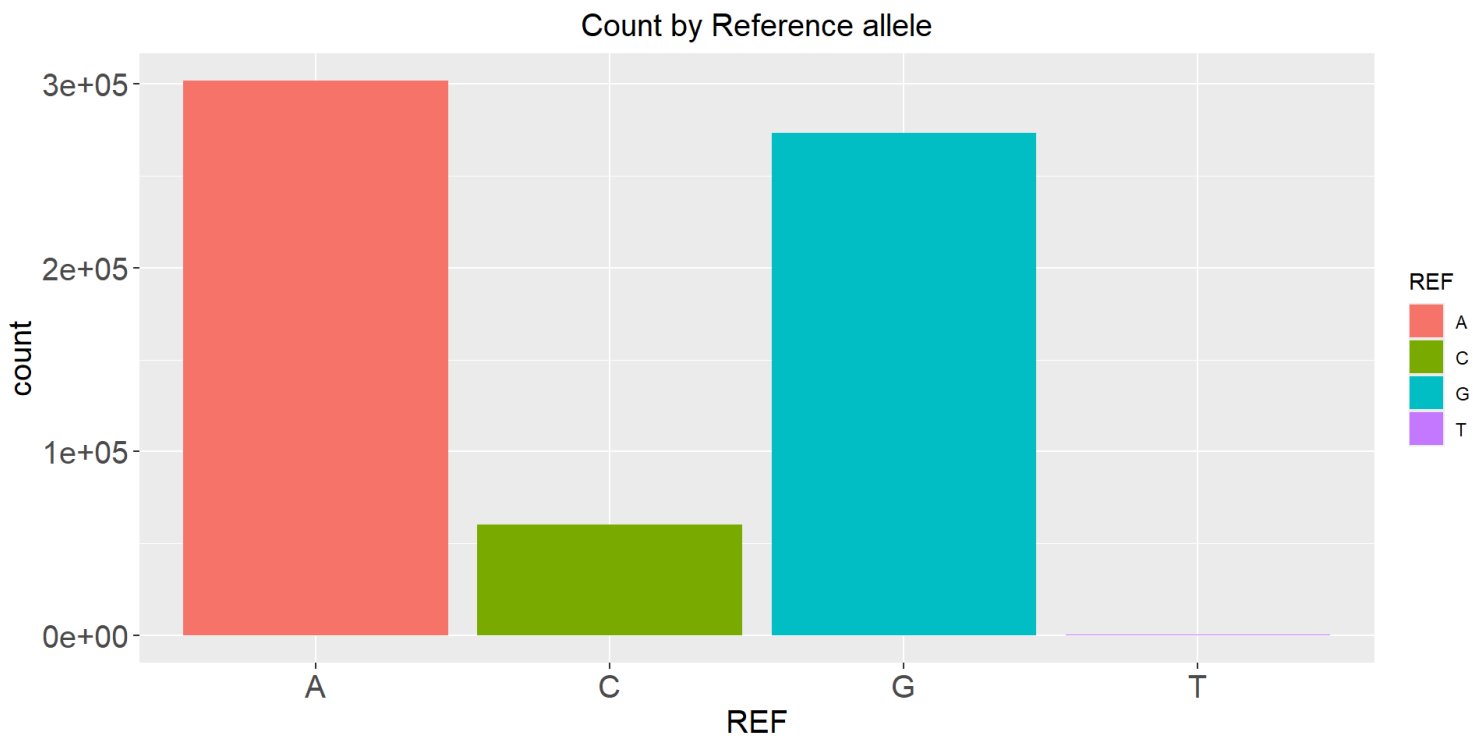
```r
set.seed(520)
map_df <- data.frame(cat_map, eff= rnorm(635732, 0.25, 0.012))
names(map_df) <- c("CHROM", "POS1", "POS2", "Allele", "Eff")
map_df[c("REF","ALT")] <- str_split_fixed(map_df$Allele, "/", 2)

head(map_df)
```

```
  CHROM   POS1   POS2 Allele       Eff REF ALT
1     1  16947  16947    C/A 0.2370945   C   A
2     1  36337  36337    A/G 0.2345670   A   G
3     1  78655  78655    A/G 0.2354766   A   G
4     1  83412  83412    G/A 0.2510536   G   A
5     1  89725  89725    A/G 0.2441561   A   G
6     1 100260 100260    G/A 0.2787393   G   A
```

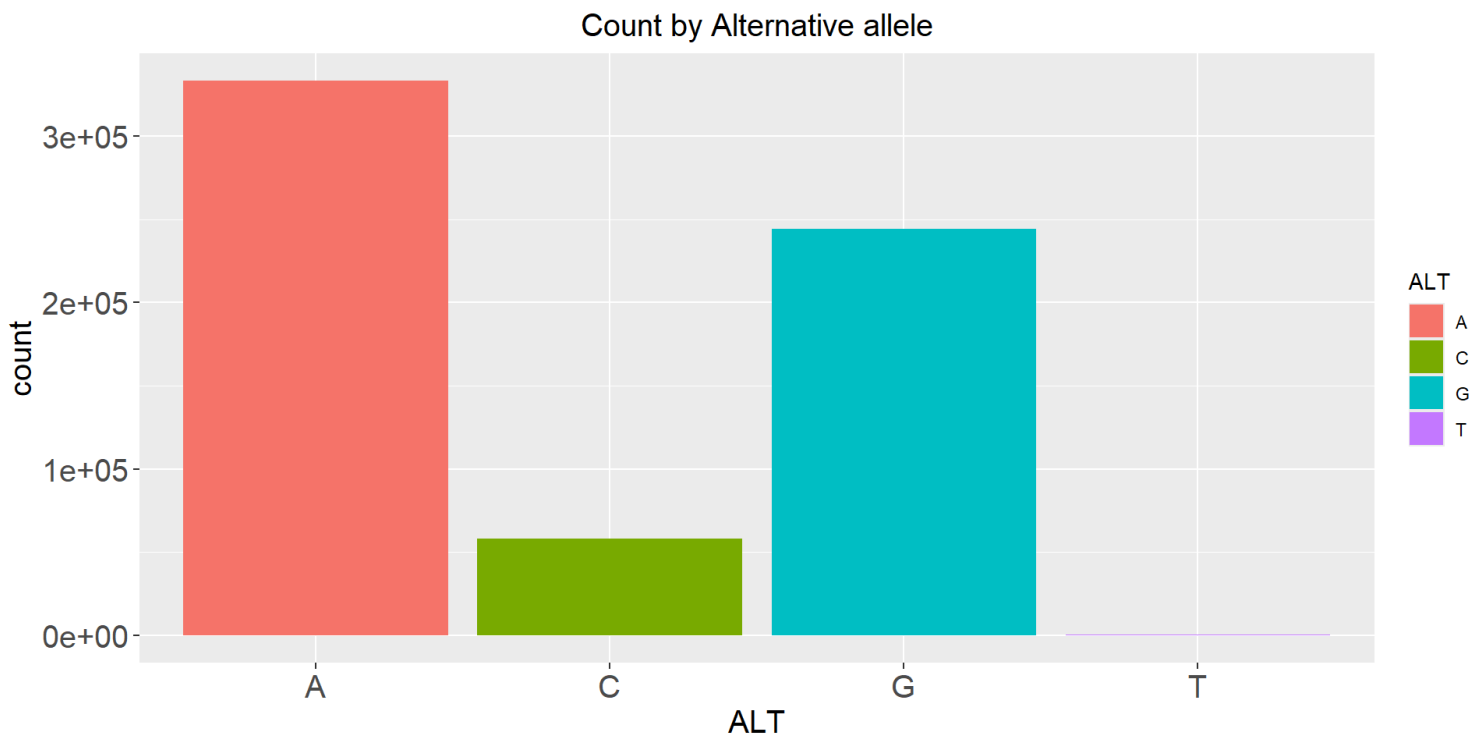
```r
p1 <- ggplot(map_df, aes(REF, Eff))+
  geom_boxplot()+
  ggtitle("Marker effects by allele")+
  theme(plot.title = element_text(hjust = 0.5, size=15),#centre plot title&
        axis.text = element_text(size = 15), #increase axis text size
        axis.title = element_text(size = 15), #increase axis title sze
        plot
        )
p1
```



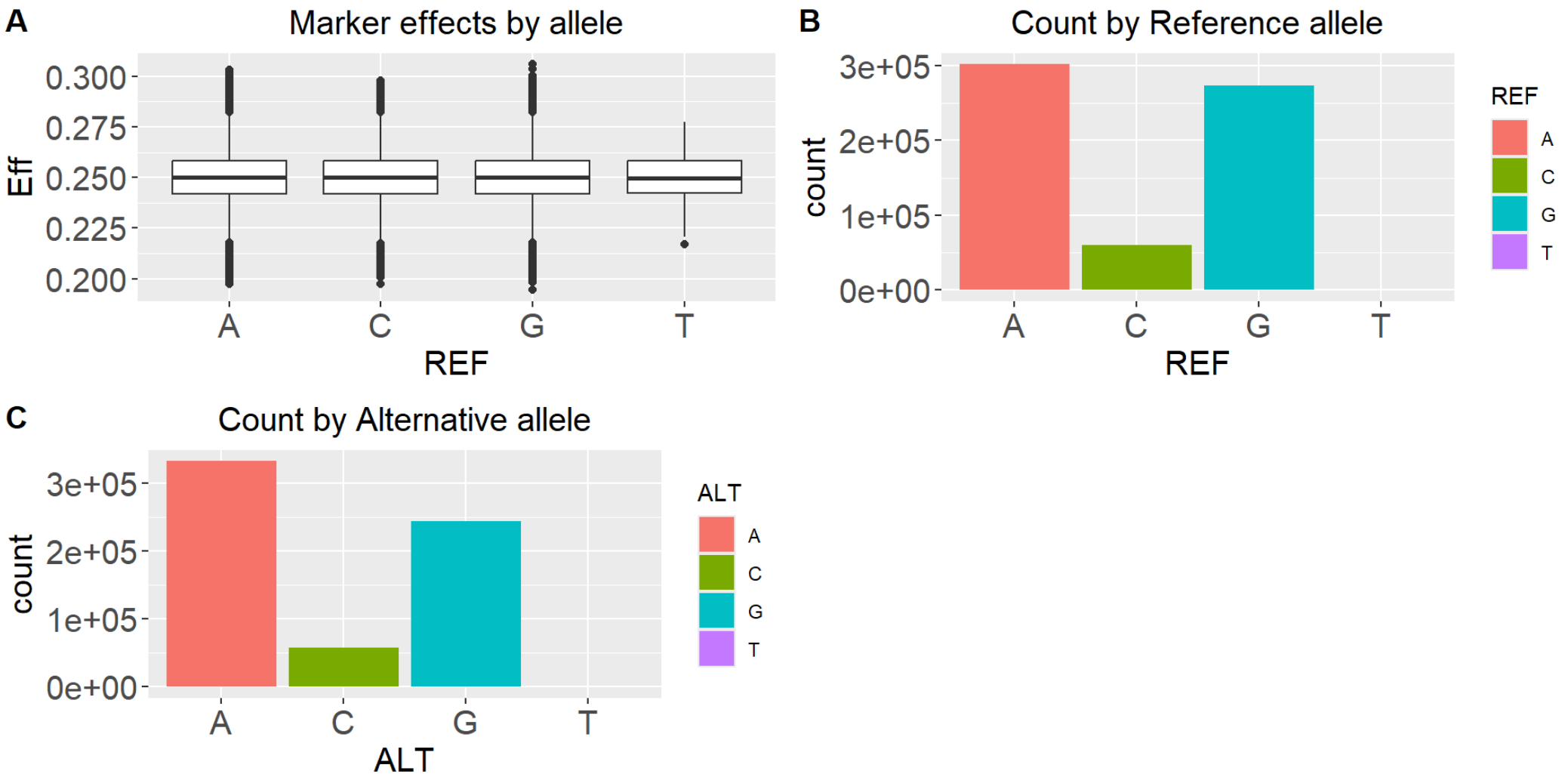
```r
p2 <- ggplot(map_df)+
  geom_bar(mapping= aes(REF, fill = REF))+
  ggtitle("Count by Reference allele")+
  theme(plot.title = element_text(hjust = 0.5, size=15),#centre plot title&
        axis.text = element_text(size = 15), #increase axis text size
        axis.title = element_text(size = 15), #increase axis title sze
        plot
        )
p2
```



```r
p3 <- ggplot(map_df)+
  geom_bar(mapping= aes(ALT, fill = ALT))+
  ggtitle("Count by Alternative allele")+
  theme(plot.title = element_text(hjust = 0.5, size=15),#centre plot title&
        axis.text = element_text(size = 15), #increase axis text size
        axis.title = element_text(size = 15), #increase axis title sze
        plot
        )
p3
```

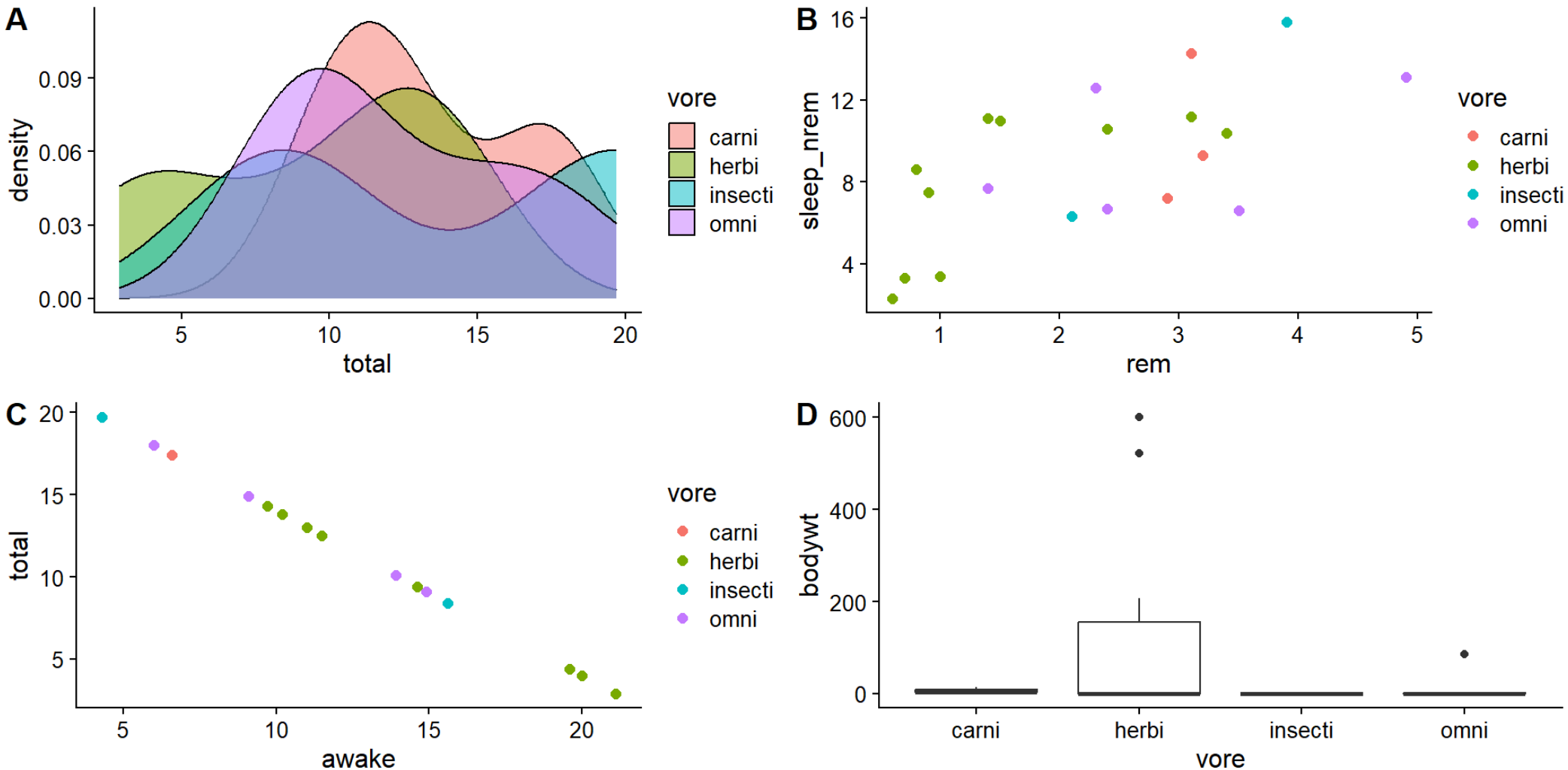


```r
library(cowplot)

plot_grid(p1, p2, p3, labels = c("A", "B", "C"))
```



```r
S1 <- sleep_df |> ggplot(aes(total, fill = vore))+
    geom_density(alpha=0.5)+
    theme_cowplot(12)

S2 <- sleep_df |> mutate(sleep_nrem = (1-(rem/total))*total)|> ggplot(aes(r
    geom_point(size=2)+
    theme_cowplot(12)

S3 <- sleep_df |> ggplot(aes(awake, total, color = vore))+
    geom_point(size=2)+
    theme_cowplot(12)

S4 <- sleep_df |> ggplot(aes(vore,bodywt))+
    geom_boxplot()+
    theme_cowplot(12)
```

```r
plot_grid(S1, S2, S3, S4, labels = c("A", "B", "C", "D"))
```



# Tables

Tables are another of displaying or presenting data. The *knitr* package is useful but if you want some elegance, the *flextable* package comes handy.

```
sleep_tbl <- sleep_df |>
  group_by(vore, conservation) |>
  summarise(avg_sleep=mean(total),
            avg_awake=mean(awake), avg_bodywt=mean(bodywt))
head(sleep_tbl)
```

```
# A tibble: 6 × 5
# Groups:   vore [2]
  vore  conservation avg_sleep avg_awake avg_bodywt
  <chr> <chr>            <dbl>     <dbl>      <dbl>
1 carni domesticated     11.3       12.7      8.65
2 carni lc               17.4        6.6      3.5
3 herbi domesticated      7.44      16.6    225.
4 herbi en               14.3        9.7      0.12
5 herbi lc               13.4       10.6      0.211
6 herbi nt               12.5       11.5      0.022
```

# Kable

```
library(kableExtra)
knitr::kable(sleep_tbl) |>
  kable_styling(font_size = 20)
```

| vore | conservation | avg_sleep | avg_awake | avg_bodywt |
|---|---|---|---|---|
| carni | domesticated | 11.300 | 12.700 | 8.6500 |
| carni | lc | 17.400 | 6.600 | 3.5000 |
| herbi | domesticated | 7.440 | 16.560 | 224.9296 |
| herbi | en | 14.300 | 9.700 | 0.1200 |
| herbi | lc | 13.400 | 10.600 | 0.2105 |
| herbi | nt | 12.500 | 11.500 | 0.0220 |
| herbi | vu | 4.400 | 19.600 | 207.5010 |
| insecti | lc | 14.050 | 9.950 | 0.0490 |
| omni | domesticated | 9.100 | 14.900 | 86.2500 |
| omni | lc | 13.025 | 10.975 | 0.6235 |

# Flextable

```
1  library(flextable)
2
3  ft1 <- flextable(sleep_tbl)
4
5  set_table_properties(ft1, layout = "autofit")
```

| vore | conservation | avg_sleep | avg_awake | avg_bodywt |
|---|---|---|---|---|
| carni | domesticated | 11.300 | 12.700 | 8.6500 |
| carni | lc | 17.400 | 6.600 | 3.5000 |
| herbi | domesticated | 7.440 | 16.560 | 224.9296 |
| herbi | en | 14.300 | 9.700 | 0.1200 |
| herbi | lc | 13.400 | 10.600 | 0.2105 |
| herbi | nt | 12.500 | 11.500 | 0.0220 |
| herbi | vu | 4.400 | 19.600 | 207.5010 |
| insecti | lc | 14.050 | 9.950 | 0.0490 |
| omni | domesticated | 9.100 | 14.900 | 86.2500 |
| omni | lc | 13.025 | 10.975 | 0.6235 |

# References

- cowplot
- flextable

# Labs

## Load packages

```r
pkgs <- c("tidyverse", "dplyr", "readr", "magrittr", "ggplot2",
          "cowplot", "GGally", "knitr","kableExtra", "data.table",
          "skimr", "flextable", "Metrics", "ggpubr", "devtools")

for (i in seq_along(pkgs)){
  pkg <- pkgs[[i]]
  if(pkg%in%rownames(installed.packages())){
    cat("Available:", pkg, "\n")
  }
  else{
    cat("Not available:",pkg,">> now installing...", "\n")
    install.packages(pkg, repos =  "https://cloud.r-project.org")
    cat("Now available:", pkg, "\n")
  }

  library(pkg, character.only = TRUE)
  cat("Package loaded:", pkg,"\n")
}
```